UNIVERSIDAD EAFIT

Internet: Arquitectura y Protocolos

Plataforma Distribuida de Telemetría y Gestión de Infraestructura Inteligente

Informe técnico

Integrantes

Juan José Díaz Ramírez

Daniel Arcila

Mariana Sánchez

Docente

Albert Montoya

Medellín, Colombia

Semestre 2026-2 · 20 de septiembre de 2026

Tabla de contenido

Tabla de contenido 2
1. Introducción y descripción del problema 4
1.1. Alcance y reparto del trabajo 4
2. Arquitectura del sistema 4
2.1. Justificación de los protocolos de transporte 5
3. Diseño y especificación del protocolo 6
3.1. Mensajes de telemetría sobre UDP 6
3.2. Comandos de consulta sobre TCP 7
3.3. Intercambio de ejemplo 7
3.4. Semántica del estado 7
4. Implementación de sockets TCP y UDP 8
4.1. Socket UDP del servidor 8
4.2. Socket TCP del servidor 8
4.3. Sockets en los clientes 9
4.4. Correspondencia con las operaciones exigidas 9
5. Concurrencia y programación de clientes y servidor 10
5.1. Estrategia adoptada 10
5.2. Protección del estado compartido 10
5.3. Contadores sin bloqueo 11
5.4. Robustez frente a desconexiones 11
5.5. Concurrencia en los clientes 11
6. Despliegue en nube, DNS y Docker 12
6.1. Resolución de nombres 13
6.2. Seguridad de acceso 13
6.3. Contenedor Docker 13
7. Análisis de tráfico con Wireshark 14
7.1. Condiciones de la captura 14
7.2. Datagrama de telemetría sobre UDP 14
7.3. Establecimiento y cierre de una conexión TCP 15
7.4. El protocolo de aplicación sobre TCP 16
7.5. Tráfico HTTP del panel web 17
7.6. Jerarquía de protocolos 18
7.7. Conversación entre los extremos 19

7.8. Correspondencia con las capas del modelo TCP/IP 20
8. Pruebas y resultados 20
8.1. Prueba de mensajes transmitidos, recibidos y perdidos 21
8.2. Generación de alertas 21
8.3. Recuperación ante desconexión 21
9. Problemas encontrados y soluciones implementadas 22
9.1. Cambio de la dirección pública de la instancia 22
9.2. Acceso denegado desde equipos no autorizados 22
9.3. Captura de tráfico sobre la interfaz equivocada 22
9.4. Identificadores incompletos en las consultas 22
9.5. Terminadores de línea del script de arranque 22
9.6. Salida del servidor no visible en los registros del contenedor 22
9.7. Binarios compilados en el control de versiones 23
10. Conclusiones 23
11. Evidencias individuales 23
11.1. Juan José Díaz Ramírez 24
11.2. Daniel Arcila 24
11.3. Mariana Sánchez 29

1. Introducción y descripción del problema

Las organizaciones que administran infraestructura distribuida geográficamente —plantas industriales, centros de datos, redes de sensores ambientales— enfrentan un problema común: los dispositivos que deben vigilar se encuentran en ubicaciones remotas, mientras que las decisiones se toman de forma centralizada. Se requiere entonces un mecanismo que transporte mediciones desde los dispositivos hasta un punto central, y que permita a los operadores consultar el estado del sistema y recibir avisos cuando algo se sale de lo normal.

Este proyecto implementa una plataforma que resuelve ese problema mediante una aplicación distribuida accesible a través de Internet. Los dispositivos se simulan con nodos de telemetría que reportan periódicamente cuatro variables: temperatura, humedad, consumo energético y vibración. Un servidor central, escrito en C y desplegado en la nube dentro de un contenedor Docker, recibe esas mediciones, detecta valores anómalos, genera alertas y responde consultas de los operadores.

El propósito académico del trabajo no se limita a lograr que varios programas intercambien información. Se trata de recorrer de forma verificable la cadena completa que va desde la aplicación hasta el tráfico observable en la red: aplicación, sockets, protocolos de transporte, direccionamiento, resolución de nombres, Internet, nube, contenedores y análisis del tráfico resultante.

1.1. Alcance y reparto del trabajo

El sistema se compone de tres elementos: cinco o más nodos de telemetría, un servidor central y clientes operadores, complementados por un servicio web de consulta. La comunicación emplea dos protocolos de transporte con funciones deliberadamente diferenciadas y un protocolo propio de capa de aplicación, diseñado por el equipo y basado en texto.

El desarrollo del servidor en C, los nodos y el cliente operador estuvo a cargo de Daniel Arcila y Mariana Sánchez. El despliegue en AWS, la configuración de Docker y las pruebas funcionales sobre la instancia fueron realizados por Daniel Arcila. La captura y el análisis del tráfico de red con Wireshark, documentados en la sección 7, corresponden a Juan José Díaz Ramírez.

2. Arquitectura del sistema

La arquitectura responde a un modelo cliente-servidor con un punto central de concentración. La Figura 1 presenta la disposición de los componentes, los protocolos que los comunican y los puertos empleados.

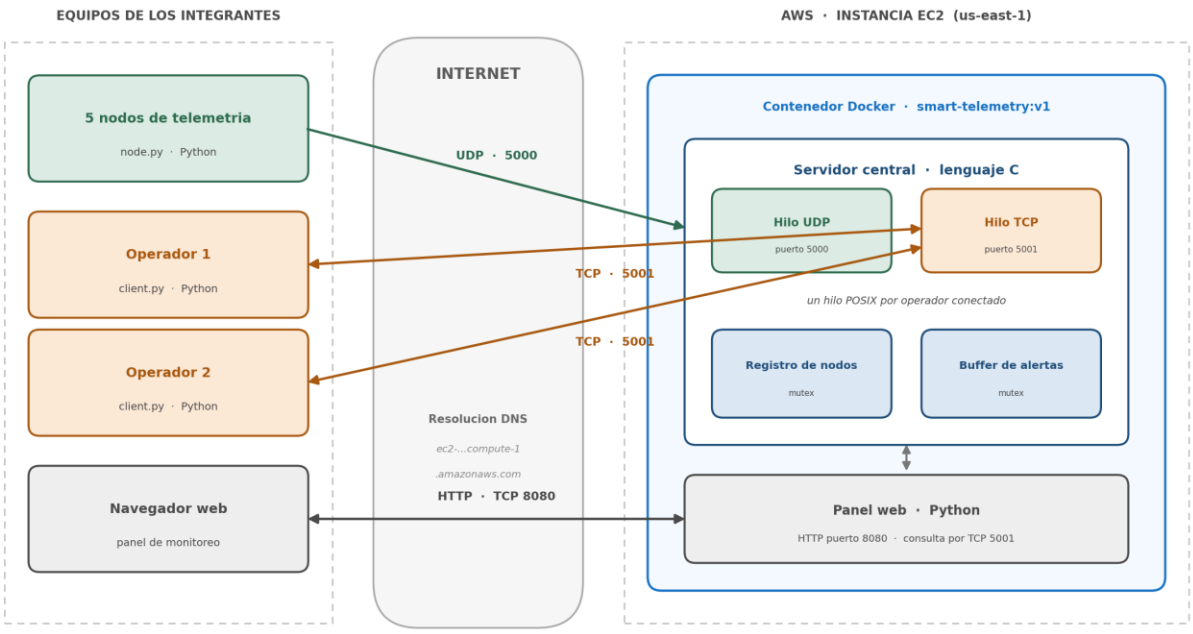


Figura 1. Arquitectura del sistema. Los nodos y operadores se ejecutan en los equipos de los integrantes; el servidor central y el panel web residen en un contenedor Docker sobre una instancia EC2. Todo el tráfico atraviesa Internet y el servidor se localiza por nombre DNS.

Los nodos de telemetría, del lado izquierdo, son procesos Python independientes que simulan dispositivos IoT. Cada uno posee un identificador único, genera valores para las cuatro variables y los transmite cada cinco segundos. Antes de emitir resuelve el nombre del servidor mediante DNS, de modo que el código no contiene ninguna dirección IP fija.

El servidor central, en el extremo derecho, ejecuta dos hilos permanentes. El hilo UDP atiende el puerto 5000 y procesa las mediciones entrantes; el hilo TCP atiende el puerto 5001 y responde las consultas de los operadores, creando un hilo adicional por cada cliente que se conecta. Ambos comparten dos estructuras de datos —el registro de nodos y el búfer de alertas— protegidas por exclusiones mutuas independientes.

Los operadores son clientes de consola que consultan el estado del sistema. El panel web, alojado en el mismo contenedor, ofrece la misma información en el navegador; no accede al estado por un camino privilegiado, sino que abre conexiones TCP contra el puerto 5001 y emplea exactamente los mismos comandos que el cliente operador.

2.1. Justificación de los protocolos de transporte

La telemetría se transmite por UDP porque se trata de un flujo periódico y tolerante a pérdidas: si una medición no llega, cinco segundos después arriba otra más reciente que la sustituye, de modo que la información nunca queda desactualizada. Establecer una conexión para cada medición supondría un costo desproporcionado frente al beneficio.

Las consultas de los operadores, en cambio, se transmiten por TCP. Un comando debe llegar íntegro y su respuesta debe recibirse completa y en orden, pues una respuesta truncada resultaría indistinguible de un error. El análisis de tráfico de la sección 7 cuantifica el costo de esa garantía sobre tráfico real.

3. Diseño y especificación del protocolo

El sistema emplea un protocolo propio de capa de aplicación basado en texto ASCII, con dos familias de mensajes que viajan sobre transportes distintos. No se adoptó HTTP, MQTT ni ningún otro protocolo existente para la comunicación principal, tanto por requisito del proyecto como porque un formato simple permite que el servidor en C lo analice sin bibliotecas externas.

El formato se apoya en tres convenciones. Los campos se separan con la barra vertical, carácter que no aparece en ningún valor legítimo. Los decimales usan punto, de modo que las funciones de conversión de C y Python los interpreten igual. Y el identificador de la operación ocupa siempre el primer campo, lo que permite decidir el tipo de mensaje leyendo su inicio.

Una consecuencia deliberada de usar texto plano es que los mensajes resultan legibles en un analizador de red sin decodificación alguna, propiedad que se aprovecha en el análisis de la sección 7.

3.1. Mensajes de telemetría sobre UDP

Los nodos envían sus mediciones al puerto UDP 5000. Cada datagrama transporta un mensaje completo con la estructura siguiente:

```
TELEMETRY|<id>|TEMP|<valor>|HUM|<valor>|ENERGY|<valor>|VIB|<valor>
```

Un ejemplo real, tomado de la captura de tráfico documentada más adelante:

```
TELEMETRY|NODE04|TEMP|33.72|HUM|71.86|ENERGY|177.42|VIB|4.58
```

El mensaje consta de exactamente diez campos y el servidor rechaza cualquier variación. La Tabla 1 recoge las reglas de validación aplicadas por el analizador antes de aceptar una medición.

Tabla 1. Reglas de validación de los mensajes de telemetría.

Regla	Comprobación	Código
Longitud	Entre 1 y 1023 bytes	`protocol.c:28`
Separadores	Sin barra inicial ni final, sin barras consecutivas	`protocol.c:28`
Número de campos	Exactamente 10	`protocol.c:32`
Etiquetas	TELEMETRY, TEMP, HUM, ENERGY y VIB en posiciones fijas	`protocol.c:37`
Identificador	De 1 a 31 caracteres alfanuméricos, guion bajo o guion	`protocol.c:45`
Valores	Finitos y no negativos	`protocol.c:14`
Humedad	No superior a 100	`protocol.c:54`

El orden de los campos es fijo y no se admiten permutaciones. Un datagrama que incumpla cualquier regla se descarta e incrementa un contador de mensajes inválidos, consultable mediante el comando

GET_STATS. El servidor no emite respuesta alguna ante un mensaje de telemetría, sea válido o no, de forma coherente con el carácter no orientado a conexión del transporte.

3.2. Comandos de consulta sobre TCP

Los operadores consultan el sistema a través del puerto TCP 5001. Cada conexión transporta un único comando terminado en salto de línea; el servidor envía la respuesta y cierra. El límite por comando es de 127 bytes.

Tabla 2. Comandos del protocolo y sus respuestas.

Comando	Respuesta correcta	Error posible
GET_NODES	OK|NODES|<n>|<id>...	—
GET_STATUS|<id>	OK|NODE|<id>|TEMP|<n>|HUM|<n>|ENERGY|<n>|VIB|<n>|ACTIVE|<0/1>	ERROR|NODE_NOT_FOUND
GET_LATEST	Igual al anterior, con prefijo OK|LATEST	ERROR|NO_TELEMETRY
GET_ALERTS	OK|ALERTS|<n>|<id>,<tipo>,<valor>...	—
GET_SYSTEM_STATUS	OK|SYSTEM|NODES|<n>|ALERTS|<n>	—
GET_STATS	OK|STATS|RECEIVED|<n>|INVALID|<n>	—

Toda respuesta comienza por OK o por ERROR, lo que permite al cliente decidir cómo interpretarla leyendo el primer campo. El protocolo define seis códigos de error. NODE_NOT_FOUND se emite al consultar un identificador no registrado; NO_TELEMETRY cuando se solicita la última medición y aún no se ha recibido ninguna; UNKNOWN_COMMAND ante un comando no reconocido; EMPTY_COMMAND ante una línea vacía; COMMAND_TOO_LONG al superar los 127 bytes; e INCOMPLETE_COMMAND cuando la conexión se cierra o expira antes de que llegue el terminador.

Los cuatro últimos se generan antes siquiera de intentar interpretar el comando, mientras que los dos primeros dependen del estado del servidor.

3.3. Intercambio de ejemplo

El siguiente diálogo, reconstruido a partir de la captura de tráfico real, ilustra una consulta completa:

```
Operador -> Servidor:  GET_NODES
Servidor -> Operador:  OK|NODES|5|NODE01|NODE02|NODE03|NODE04|NODE05
```

Y un caso de error, observado durante las pruebas al introducir un identificador incompleto:

```
Operador -> Servidor:  GET_STATUS|3
Servidor -> Operador:  ERROR|NODE_NOT_FOUND
```

3.4. Semántica del estado

Dos reglas del protocolo dependen del estado que mantiene el servidor y afectan a la interpretación de las respuestas. Un nodo se considera activo si reportó en los últimos quince segundos; el comando GET_NODES lista únicamente los activos, mientras que GET_SYSTEM_STATUS cuenta todos los registrados desde el arranque. Un nodo detenido desaparece del primer listado pero sigue contando en el segundo.

Las alertas se conservan en un búfer circular de cien posiciones. Cuando se llena, cada alerta nueva sobrescribe la más antigua, de modo que GET_ALERTS devuelve siempre las cien más recientes en orden cronológico. Durante las pruebas con cinco nodos activos ese búfer se renovaba por completo en menos de dos minutos. Todo el estado reside en memoria y se pierde al reiniciar el servidor o el contenedor.

4. Implementación de sockets TCP y UDP

4.1. Socket UDP del servidor

El servidor crea el socket de telemetría mediante una llamada a socket con la familia AF_INET y el tipo SOCK_DGRAM, que selecciona el servicio de datagramas. A continuación asocia la dirección con bind, empleando INADDR_ANY para aceptar tráfico dirigido a cualquier interfaz del equipo, condición necesaria para que el contenedor reciba paquetes procedentes de Internet.

No existen llamadas a listen ni a accept, porque UDP no establece conexiones: el socket queda directamente en disposición de recibir. La recepción se realiza en un bucle infinito mediante recvfrom, función que además entrega la dirección del emisor, lo que permite registrar en consola la dirección de origen de cada medición.

El tratamiento de errores distingue dos situaciones. Los fallos en la creación del socket o en el bind son irrecuperables y terminan el proceso con un mensaje de diagnóstico. En cambio, un error puntual en recvfrom no detiene el servidor: el código lo reporta y continúa el bucle, de modo que un paquete problemático no interrumpe el servicio para los demás nodos.

Existe además una comprobación defensiva. Antes de tratar el contenido recibido como una cadena de texto, el servidor descarta los datagramas que ocupan todo el búfer —señal de posible truncamiento— y los que contienen bytes nulos intercalados, que romperían el análisis posterior. Solo tras superar esa comprobación se añade el terminador nulo y se invoca al analizador del protocolo.

4.2. Socket TCP del servidor

El socket de consultas se crea con el tipo SOCK_STREAM. Antes de asociarlo se activa la opción SO_REUSEADDR, que permite reutilizar el puerto inmediatamente tras un reinicio del servidor sin esperar a que expiren las conexiones en estado TIME_WAIT. Esta opción resultó especialmente útil durante las pruebas de reinicio del contenedor.

La secuencia continúa con bind y listen, este último con una cola de diez conexiones pendientes. El servidor entra entonces en un bucle permanente de accept, que bloquea hasta que un operador solicita conexión y devuelve un descriptor nuevo dedicado a ese cliente.

La recepción del comando se realiza byte a byte hasta encontrar el salto de línea, descartando los retornos de carro para tolerar clientes que terminen las líneas al estilo de Windows. Este enfoque evita tener que gestionar la fragmentación de TCP: dado que el protocolo delimita los comandos por salto de línea y no por longitud, leer hasta el delimitador garantiza procesar un comando completo con independencia de cómo se hayan repartido los bytes entre segmentos.

Para impedir que un cliente que abra una conexión y no envíe nada mantenga un hilo bloqueado indefinidamente, se establece un tiempo máximo de espera de tres segundos mediante SO_RCVTIMEO. Si el plazo expira sin haber recibido el terminador, el servidor responde con el error INCOMPLETE_COMMAND y cierra.

El envío de la respuesta se realiza dentro de un bucle que contabiliza los bytes efectivamente transmitidos y reintenta hasta completar el total. Esta precaución es necesaria porque send puede transmitir menos bytes de los solicitados cuando el búfer del sistema está lleno, situación verosímil con respuestas extensas como la de GET_ALERTS, que llegó a ocupar cerca de tres mil bytes durante las pruebas. Finalmente el descriptor se cierra.

4.3. Sockets en los clientes

Los nodos de telemetría comienzan resolviendo el nombre del servidor mediante gethostbyname. Esta llamada materializa el requisito de localizar el servidor por DNS: el código no contiene ninguna dirección, y el nombre se recibe como argumento de línea de órdenes. Un fallo de resolución se captura como excepción específica y el nodo termina con un mensaje explicativo en lugar de abortar con una traza.

El socket se crea con el tipo de datagramas y el envío se realiza con sendto, indicando destino en cada llamada, puesto que un socket UDP no conserva un interlocutor fijo. El cierre se garantiza mediante un bloque finally, que se ejecuta incluso cuando el nodo se detiene con una interrupción de teclado.

El cliente operador crea un socket de flujo, fija un tiempo máximo de espera de cinco segundos y establece la conexión con connect. El envío emplea sendall, que a diferencia de send garantiza la transmisión completa del comando. La recepción se realiza en un bucle que acumula fragmentos hasta que recv devuelve una cadena vacía, condición que en TCP señala que el otro extremo ha cerrado su lado de la conexión; ese es precisamente el mecanismo por el que el cliente detecta el final de la respuesta, dado que el protocolo no incluye un delimitador explícito de fin de mensaje.

El tratamiento de errores del cliente distingue tres casos con mensajes específicos: conexión rechazada cuando el servidor no está escuchando, fallo de resolución cuando el DNS no responde, y error genérico de comunicación para el resto. En todos los casos el cliente informa y regresa al menú en lugar de terminar.

4.4. Correspondencia con las operaciones exigidas

La Tabla 3 localiza en el código cada una de las operaciones de sockets que el proyecto debe evidenciar.

Tabla 3. Ubicación de las operaciones de sockets en el código fuente.

Operación	Servidor UDP	Servidor TCP	Clientes
Creación	`udp_server.c:57`	`tcp_server.c:123`	`node.py:64` · `client.py:13`
Asociación y escucha	`udp_server.c:111`	`tcp_server.c:131`	—
Aceptación de clientes	no aplica	`tcp_server.c:141`	—
Conexión	no aplica	no aplica	`client.py:16`

Operación	Servidor UDP	Servidor TCP	Clientes
Envío	no aplica	`tcp_server.c:112`	`node.py:90` · `client.py:47`
Recepción	`udp_server.c:152`	`tcp_server.c:94`	`client.py:51`
Cierre	`udp_server.c:255`	`tcp_server.c:117`	`node.py:112` · `client.py:68`
Manejo de errores	`udp_server.c:163`	`tcp_server.c:102`	`client.py:22`

La llamada a close del socket UDP está escrita tras el bucle de recepción, que es infinito, por lo que en la práctica no llega a ejecutarse: el descriptor lo libera el sistema operativo al terminar el proceso. Es aceptable en un servicio cuyo ciclo de vida coincide con el del programa, pero se señala por rigor. El cierre efectivo y verificable en cada operación corresponde al socket TCP de cada cliente, que sí se cierra explícitamente al completar la respuesta.

5. Concurrencia y programación de clientes y servidor

5.1. Estrategia adoptada

El servidor atiende simultáneamente dos tipos de tráfico con requisitos distintos: un flujo continuo de datagramas procedente de cinco o más nodos, y consultas esporádicas de varios operadores. La estrategia elegida fue el uso de hilos POSIX, en dos niveles.

En el primer nivel, la función principal crea sendos hilos permanentes, uno por cada servicio. Ambos ejecutan bucles infinitos sobre sus respectivos sockets, de modo que la recepción de telemetría y la atención de operadores progresan de forma independiente: una consulta TCP lenta no retrasa el procesamiento de mediciones, y viceversa. El hilo principal queda bloqueado esperando su terminación, manteniendo vivo el proceso.

En el segundo nivel, el hilo TCP crea un hilo adicional por cada operador que se conecta, y lo marca como desvinculado para que el sistema libere sus recursos automáticamente al terminar, sin que nadie tenga que recogerlo. Gracias a este esquema el servidor atiende un número arbitrario de operadores concurrentes, muy por encima de los dos que exige el proyecto.

Se descartó el uso de procesos independientes porque los componentes del servidor comparten el mismo estado —el registro de nodos y el historial de alertas—, y con procesos habría sido necesario recurrir a memoria compartida o a algún mecanismo de comunicación entre procesos, aumentando la complejidad sin beneficio. También se descartó la multiplexación de sockets mediante select o poll, que habría evitado los hilos a costa de una máquina de estados considerablemente más difícil de leer y mantener.

5.2. Protección del estado compartido

Al compartir estado entre hilos aparece el riesgo de que dos accesos simultáneos dejen las estructuras en un estado inconsistente. El servidor lo evita con dos exclusiones mutuas independientes. La primera protege el arreglo de nodos y la última medición recibida, y la adquieren tanto el hilo UDP al registrar una medición como cualquier hilo de operador al consultar. La segunda protege el búfer circular de alertas.

Haberlas separado no es accidental: una consulta de alertas no bloquea el registro de una medición, porque cada operación toma únicamente el cerrojo que necesita.

El código emplea además un patrón de instantánea. Las funciones de consulta adquieren el cerrojo, copian los datos a un arreglo local y lo liberan de inmediato; el formateo de la respuesta se realiza después, ya fuera de la sección crítica. De este modo el tiempo durante el cual el estado permanece bloqueado se reduce al mínimo imprescindible, incluso cuando la respuesta que se va a construir es extensa.

5.3. Contadores sin bloqueo

Los contadores de mensajes recibidos e inválidos utilizan tipos atómicos de C11 en lugar de una exclusión mutua. Son valores que se incrementan en cada datagrama procesado —del orden de una vez por segundo con cinco nodos activos— y que se leen esporádicamente desde los hilos de operador. Emplear un cerrojo para proteger un simple incremento habría introducido contención innecesaria en la ruta más frecuente del servidor; las operaciones atómicas garantizan que el incremento sea indivisible sin bloquear a nadie.

Estos contadores son los que sustentan el comando GET_STATS y, con él, la prueba de tráfico UDP descrita en la sección 8.

5.4. Robustez frente a desconexiones

El servidor debe manejar desconexiones y errores sin finalizar abruptamente. Tres mecanismos lo garantizan.

El primero es la desactivación de la señal SIGPIPE al iniciar el programa. Sin esa medida, si el servidor intentara escribir en un socket cuyo extremo remoto ya se ha cerrado —situación normal cuando un operador interrumpe el cliente a mitad de una consulta—, el sistema operativo terminaría el proceso completo. Al ignorar la señal, la escritura fallida se convierte en un simple error que el bucle de envío detecta y trata.

El segundo es el tratamiento local de los errores en los bucles de servicio: tanto un fallo de recepción como uno de aceptación o de creación de hilo provocan que el servidor libere lo que corresponda y continúe con la siguiente iteración, en lugar de abortar.

El tercero es la validación estricta del protocolo. Un mensaje malformado nunca alcanza las estructuras de datos, porque el analizador lo rechaza antes; se limita a incrementar el contador de inválidos.

Estos mecanismos se verificaron durante las pruebas. Al detener los cinco nodos, el servidor continuó operativo y las consultas pasaron a reportar cero nodos activos; al reanudarlos, los cinco volvieron a registrarse sin intervención manual.

5.5. Concurrencia en los clientes

Los cinco nodos se ejecutan como procesos independientes, lanzados desde un script auxiliar. Se optó por procesos y no por hilos porque cada nodo simula un dispositivo físicamente separado, y con procesos distintos se puede detener uno solo para observar el comportamiento del servidor ante la inactividad de

un dispositivo, tal como se hizo en las pruebas de recuperación. El lanzador captura la interrupción de teclado y termina ordenadamente todos sus hijos.

El cliente operador, en cambio, es deliberadamente secuencial: presenta un menú, espera una elección y realiza una consulta cada vez. La concurrencia entre operadores se produce porque varios clientes se ejecutan simultáneamente en equipos distintos, no porque un mismo cliente emita consultas en paralelo.

6. Despliegue en nube, DNS y Docker

El servidor se desplegó sobre una instancia de Amazon EC2 dentro de un entorno AWS Academy Learner Lab. La Tabla 4 resume los datos del despliegue documentado.

Tabla 4. Datos de la instancia desplegada.

Elemento	Valor
Identificador de instancia	i-01463d9faf35ebebf
Región y zona	us-east-1 (N. Virginia), zona us-east-1f
Tipo de instancia	t3.small
Sistema operativo	Amazon Linux 2023.12.20260918
Nombre DNS público	ec2-3-236-83-176.compute-1.amazonaws.com
Dirección IPv4 pública	3.236.83.176
Imagen Docker	smart-telemetry:v1 (124 MB)
Puertos publicados	UDP 5000 · TCP 5001 · TCP 8080
Motor Docker	25.0.16

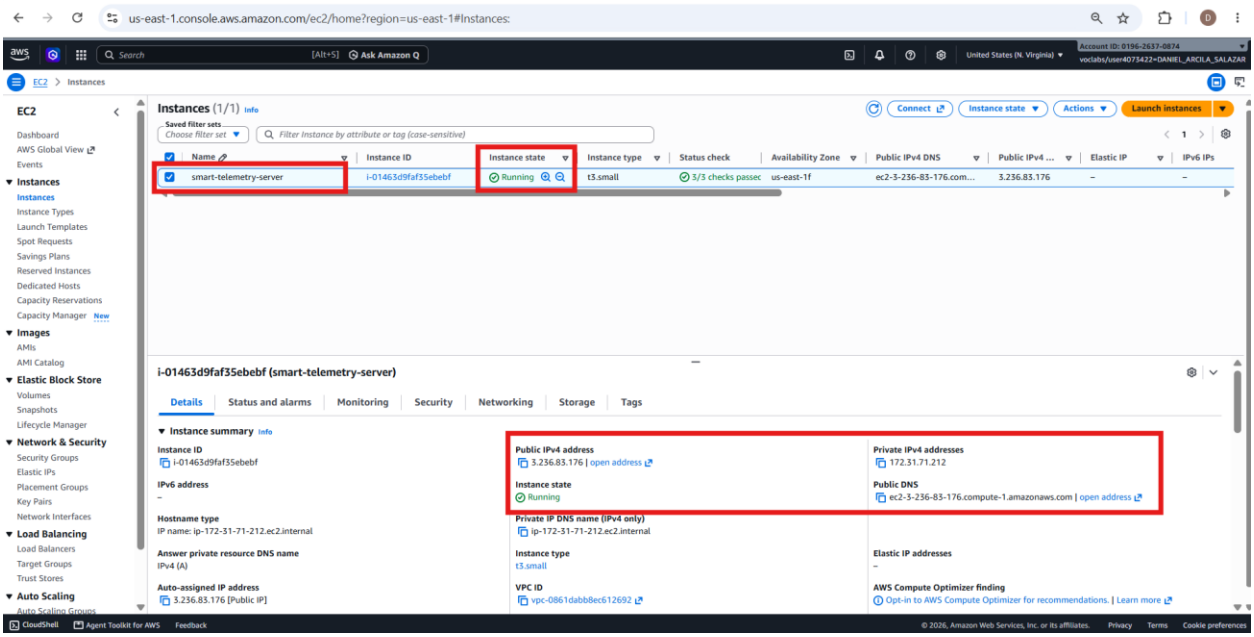


Figura 2. Instancia EC2 en ejecución, con estado Running, comprobaciones 3/3 superadas y la dirección IPv4 pública asociada.

6.1. Resolución de nombres

El requisito de no incluir direcciones fijas en el código se satisface porque tanto los nodos como el cliente operador reciben el nombre del servidor como argumento y lo resuelven en tiempo de ejecución. Durante las pruebas se empleó el nombre DNS público que AWS asigna a la instancia.

Debe señalarse una limitación de ese enfoque. La dirección pública de una instancia de laboratorio cambia al detenerla e iniciarla, y con ella el nombre DNS asociado. Durante el desarrollo se produjo efectivamente ese cambio: las primeras pruebas se realizaron contra la dirección 44.192.90.37 y las posteriores contra 3.236.83.176. Antes de ejecutar los clientes es necesario, por tanto, consultar el nombre vigente en la consola de EC2.

6.2. Seguridad de acceso

El acceso al servidor se controla mediante un grupo de seguridad que autoriza los puertos necesarios, restringidos a las direcciones públicas de los integrantes mediante máscaras de host individuales. La Figura 3 muestra las reglas configuradas.

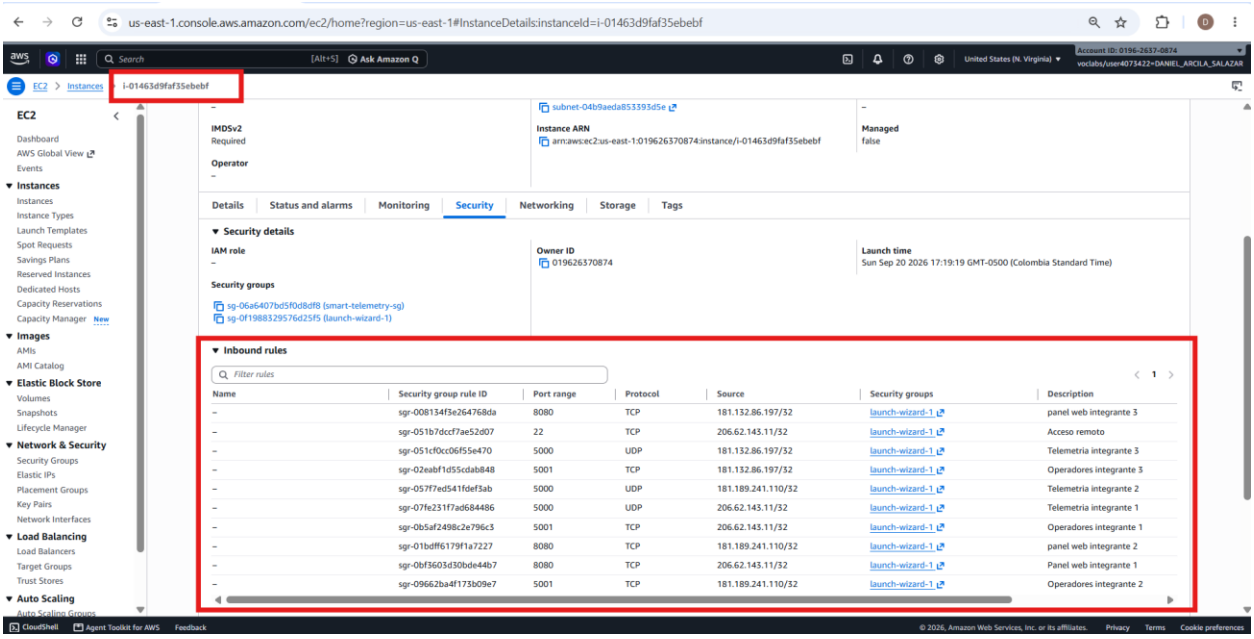


Figura 3. Grupos de seguridad asociados a la instancia y reglas de entrada para acceso remoto, telemetría, consultas de operadores y panel web.

6.3. Contenedor Docker

El servidor se ejecuta dentro de un contenedor construido en dos etapas. La primera emplea una imagen con el compilador de C y produce el binario del servidor; la segunda parte de una imagen ligera de Python e incorpora únicamente el binario compilado y el panel web, de modo que el compilador no forma parte de la imagen final. Esa separación explica que la imagen resultante ocupe 124 MB pese a incluir dos entornos de ejecución.

El punto de entrada del contenedor lanza el servidor en C y el panel web como procesos paralelos, y captura las señales de terminación para detener ambos de forma ordenada. La construcción de la imagen finalizó correctamente y el contenedor se ejecutó publicando los tres puertos del sistema, con reinicio automático configurado.

```
sudo docker build -t smart-telemetry:v1 .
sudo docker run -d --name smart-telemetry --restart unless-stopped \
  -e PYTHONUNBUFFERED=1 \
  -p 5000:5000/udp -p 5001:5001/tcp -p 8080:8080/tcp \
  smart-telemetry:v1
```

Una comprobación local dentro de la instancia confirmó que el panel web respondía con código 200, y las pruebas externas descritas en la sección 8 verificaron el acceso desde Internet.

7. Análisis de tráfico con Wireshark

7.1. Condiciones de la captura

La captura se realizó desde un equipo con Windows conectado a Internet por Wi-Fi, mientras el servidor se ejecutaba dentro del contenedor en la instancia EC2. La interfaz seleccionada fue el adaptador físico, no una interfaz virtual ni el bucle local, de modo que todos los paquetes registrados atravesaron efectivamente Internet.

Para limitar el registro al tráfico del proyecto se aplicó un filtro de captura por la dirección del servidor, que descarta cualquier paquete ajeno. La sesión duró 85,44 segundos y registró 567 paquetes con 137 176 bytes. Durante ese intervalo se ejecutaron los cinco nodos, se realizaron seis consultas desde el cliente operador y se recargó varias veces el panel web, de forma que una misma captura contiene los tres tipos de comunicación del sistema.

El equipo cliente aparece con la dirección privada 192.168.1.57, asignada por el router de la red local, y el servidor con la dirección pública 3.236.83.176. Conviene advertir que el servidor en AWS registra las peticiones procedentes de la dirección pública 181.132.86.197, distinta de la que observa Wireshark. La discrepancia no es un error: el router aplica traducción de direcciones después de que el paquete abandona la tarjeta de red, por lo que la captura local solo puede observar el direccionamiento previo a esa traducción. Es una manifestación directa del comportamiento de la capa de red en un escenario real.

7.2. Datagrama de telemetría sobre UDP

El filtro udp.port == 5000 aísla los 90 datagramas de telemetría registrados, el 15,9 % del total. Esa cifra concuerda con el comportamiento esperado: cinco nodos emitiendo una medición cada cinco segundos durante noventa producen exactamente noventa datagramas.

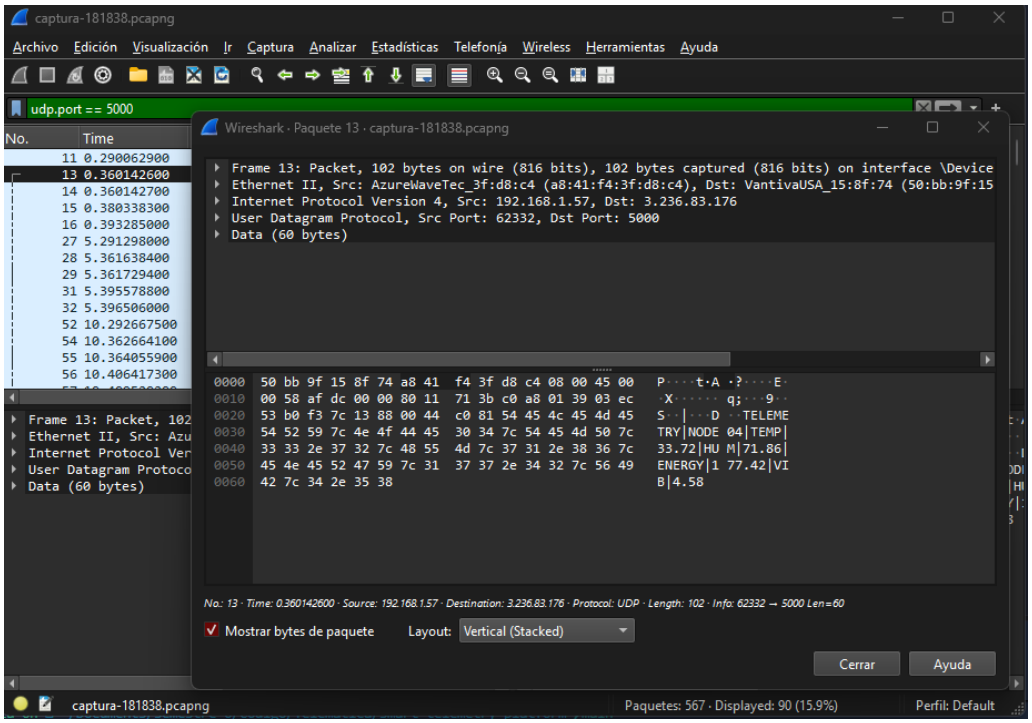


Figura 4. Detalle de un datagrama de telemetría, con las cabeceras IPv4 y UDP desplegadas y el mensaje del protocolo legible en el panel de bytes.

En la capa de red, la cabecera IPv4 identifica el origen y el destino, y su campo de protocolo vale 17, valor que el estándar asigna a UDP; es decir, la capa de red declara cuál será el protocolo de transporte antes de que este se procese. En la capa de transporte, la cabecera UDP indica el puerto de origen 62332, asignado dinámicamente al nodo, y el puerto de destino 5000, donde el servidor mantiene su socket en escucha; la cabecera declara una longitud total de 68 bytes, de los cuales 8 corresponden a la propia cabecera y 60 al contenido transportado.

En la capa de aplicación, esos 60 bytes contienen el mensaje completo del protocolo, que viaja en texto ASCII plano y puede leerse sin decodificación en la columna derecha del panel de bytes. Esta característica permite verificar visualmente la correspondencia entre la especificación del protocolo y lo que realmente circula por la red.

Debe subrayarse lo que no aparece en la figura. No hay establecimiento previo de conexión, no hay confirmación de recepción y no hay cierre. Cada datagrama constituye una transmisión completa e independiente; si uno se pierde, ningún mecanismo lo retransmite y ninguno de los dos extremos se entera. Esa es precisamente la propiedad que hace a UDP apropiado para telemetría periódica.

7.3. Establecimiento y cierre de una conexión TCP

El filtro tcp.stream == 1 aísla una única conversación contra el puerto 5001. La Figura 5 muestra el intercambio completo en diez paquetes.

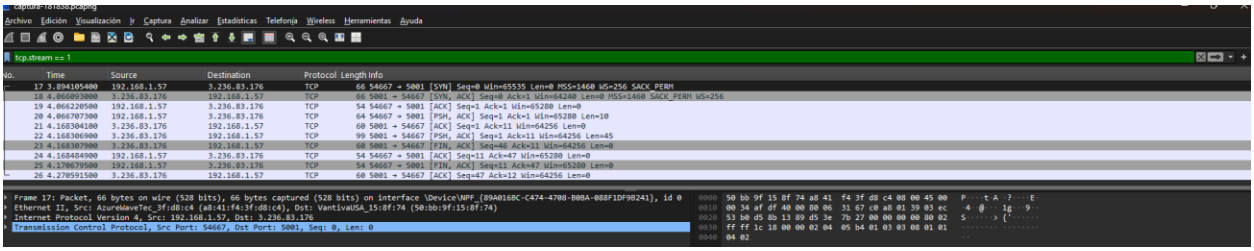


Figura 5. Ciclo de vida completo de una consulta del operador, desde el acuerdo de tres vías hasta el cierre ordenado de la conexión.

Los tres primeros paquetes constituyen el establecimiento de conexión mediante el acuerdo de tres vías: el cliente solicita la conexión con el indicador SYN y un número de secuencia inicial, el servidor responde confirmando esa solicitud y enviando la suya con SYN y ACK, y el cliente confirma la del servidor. A partir de ese punto existe un canal fiable y bidireccional entre el puerto efímero del cliente y el puerto 5001 del servidor.

Los tres paquetes siguientes transportan la información útil: uno lleva 10 bytes con el comando y su terminador, otro es una confirmación pura sin datos, y el tercero lleva 45 bytes con la respuesta. Los cuatro últimos realizan el cierre. Resulta significativo que sea el servidor quien lo inicia, lo que refleja fielmente el diseño implementado: el hilo que atiende a cada operador procesa un único comando, envía la respuesta y cierra el descriptor. Cada opción del menú del cliente abre por tanto su propia conexión, lo que explica que la captura registre seis acuerdos de tres vías para seis consultas.

7.4. El protocolo de aplicación sobre TCP

La función de seguimiento de flujo reensambla el contenido transportado por la conexión y descarta las cabeceras, de modo que se observa únicamente el intercambio tal como lo perciben las aplicaciones en cada extremo.

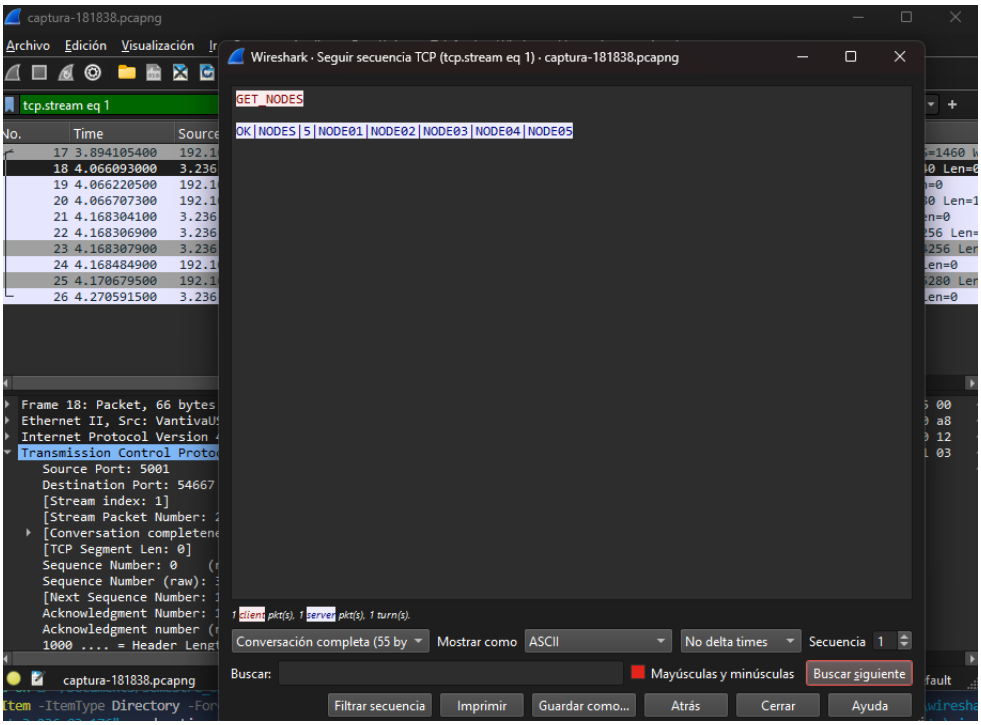


Figura 6. Reconstrucción del diálogo de la capa de aplicación, con el comando del operador y la respuesta del servidor.

La respuesta confirma que en ese instante el servidor tenía registrados cinco nodos activos, coherente con los cinco procesos en ejecución. Esta figura documenta el cumplimiento del requisito de diseñar un protocolo propio: el comando sigue la estructura especificada, con un identificador de operación, parámetros separados por barra vertical y terminación con salto de línea, y la respuesta se ajusta al mismo formato encabezada por el indicador de resultado.

7.5. Tráfico HTTP del panel web

El filtro de peticiones y respuestas HTTP aísla las 72 tramas correspondientes al servicio web, repartidas en 36 peticiones y sus 36 respuestas con contenido HTML. El elevado número se explica porque la página incluye una instrucción de recarga automática cada cinco segundos, además de las recargas manuales realizadas durante la prueba.

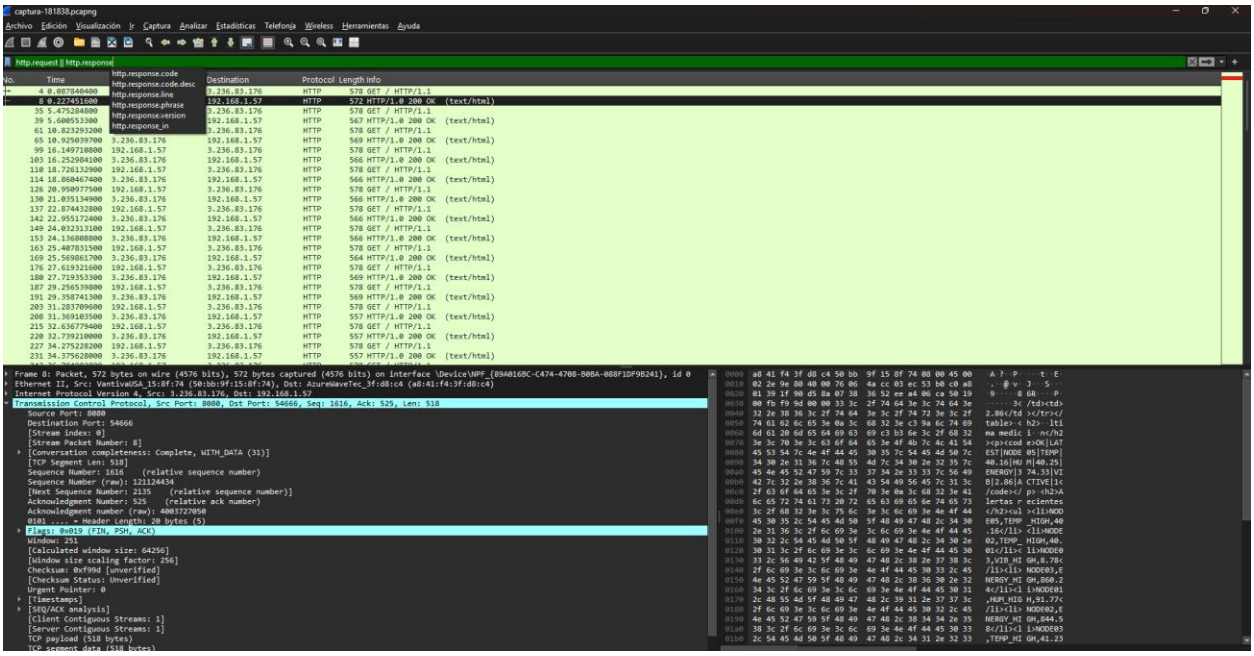
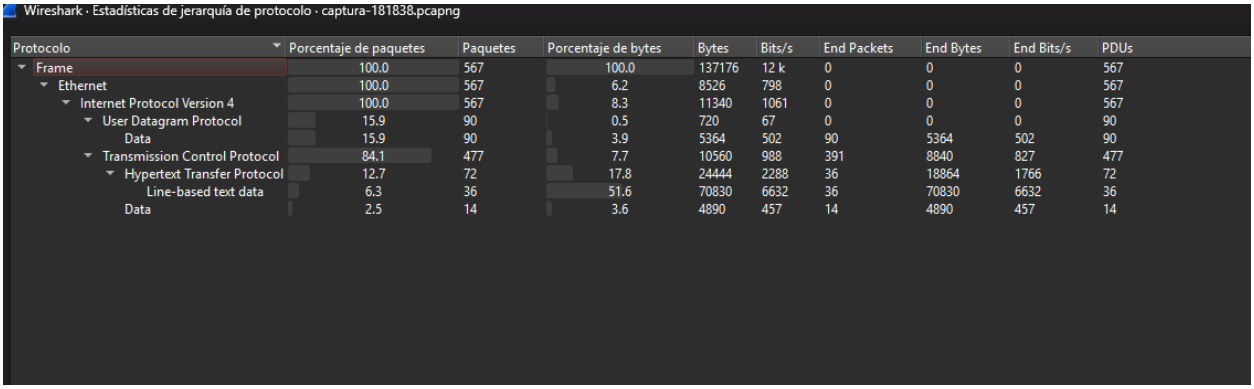


Figura 7. Peticiones y respuestas HTTP intercambiadas con el panel web, con el contenido de una respuesta visible en el panel de bytes.

El aspecto más relevante de esta figura no es el intercambio HTTP en sí, sino lo que se observa dentro del cuerpo de la respuesta. Incrustada en el código HTML que el servidor devuelve al navegador puede leerse una respuesta del protocolo propio, transportada literalmente dentro de la página. El panel no consulta una base de datos ni accede al estado del servidor por ningún mecanismo alternativo: abre una conexión TCP contra el puerto 5001, emite los mismos comandos que el cliente operador y compone la página con las respuestas obtenidas. HTTP cumple aquí exclusivamente la función de presentar información en el navegador, mientras la comunicación sustantiva continúa realizándose mediante el protocolo diseñado por el equipo.

7.6. Jerarquía de protocolos

La Figura 8 presenta el encapsulamiento completo de la pila TCP/IP tal como se observó durante la prueba.



Protocolo	Porcentaje de paquetes	Paquetes	Porcentaje de bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	567	100.0	137176	12 k	0	0	0	567
Ethernet	100.0	567	6.2	8526	798	0	0	0	567
Internet Protocol Version 4	100.0	567	8.3	11340	1061	0	0	0	567
User Datagram Protocol	15.9	90	0.5	720	67	0	0	0	90
Data	15.9	90	3.9	5364	502	90	5364	502	90
Transmission Control Protocol	84.1	477	7.7	10560	988	391	8840	827	477
Hypertext Transfer Protocol	12.7	72	17.8	24444	2288	36	18864	1766	72
Line-based text data	6.3	36	51.6	70830	6632	36	70830	6632	36
Data	2.5	14	3.6	4890	457	14	4890	457	14

Figura 8. Distribución de los 567 paquetes capturados entre los protocolos de cada capa.

Tabla 5. Reparto del tráfico por protocolo.

Protocolo	Paquetes	Porcentaje	Paquetes terminales
UDP	90	15,9 %	0
TCP	477	84,1 %	391
HTTP (sobre TCP)	72	12,7 %	36
Datos sobre TCP	14	2,5 %	14

La columna de paquetes terminales merece atención especial, pues cuantifica aquellos en los que el protocolo indicado constituye el nivel más alto alcanzado, es decir, los que no transportan datos de aplicación. En TCP ese valor asciende a 391 de 477 paquetes, un 82 % del tráfico TCP capturado: acuerdos de conexión, confirmaciones y cierres, el precio que el protocolo cobra por garantizar entrega, orden e integridad. Solo 86 paquetes TCP transportan contenido real. En UDP ese valor es cero: los noventa datagramas transportan, sin excepción, una medición.

Esta comparación constituye la justificación cuantitativa de la elección de transporte realizada en el diseño. Si la telemetría se hubiese implementado sobre TCP, cada medición habría requerido establecer y cerrar una conexión, multiplicando aproximadamente por diez el número de paquetes necesarios para entregar la misma información, a cambio de una garantía que la aplicación no necesita.

Una observación adicional respalda el cumplimiento del requisito de protocolo propio. El analizador identifica HTTP por su nombre e incluso extrae sus líneas de texto, porque se trata de un protocolo estándar cuyo disector conoce. En cambio, los catorce paquetes que transportan el protocolo del proyecto aparecen clasificados genéricamente como datos: el analizador reconoce que hay información de aplicación, pero no dispone de ningún disector capaz de interpretarla, precisamente porque el protocolo fue diseñado por el equipo y no corresponde a ningún estándar existente.

7.7. Conversación entre los extremos



Figura 9. Resumen de la única conversación IPv4 registrada, entre el equipo cliente y la instancia desplegada en la nube.

La captura registra una sola conversación a nivel de red, consecuencia del filtro aplicado. El cliente emite 310 paquetes con 40 kB, y el servidor 257 paquetes con 97 kB. El tráfico resulta marcadamente asimétrico, y en sentido inverso al que podría suponerse: el cliente emite más paquetes, porque los cinco nodos generan un flujo constante de datagramas, pero transfiere menos de la mitad de los bytes, dado que cada medición ocupa apenas 60 bytes. El servidor emite menos paquetes pero transporta más del doble de información, porque sus respuestas incluyen páginas HTML completas y listados de cien alertas.

7.8. Correspondencia con las capas del modelo TCP/IP

En la capa de aplicación operan tres protocolos distintos. El de telemetría, orientado a la emisión unidireccional de mediciones. El de consultas, orientado a la interacción entre operador y servidor. Y HTTP, empleado exclusivamente para la presentación en el navegador. Los dos primeros fueron diseñados por el equipo; el tercero se integra como capa de presentación sobre ellos.

En la capa de transporte conviven UDP y TCP con funciones deliberadamente diferenciadas. UDP transporta la telemetría en datagramas independientes sin garantía de entrega; TCP transporta las consultas y el tráfico web, estableciendo conexiones cuyo ciclo completo quedó documentado. Los puertos 5000, 5001 y 8080 identifican los tres servicios dentro del mismo equipo.

En la capa de red, IPv4 encamina todos los paquetes entre la red doméstica del cliente y el centro de datos en Virginia, empleando el campo de protocolo de su cabecera para indicar en cada caso cuál de los dos protocolos de transporte transporta. La diferencia entre la dirección privada observada localmente y la dirección pública registrada por el servidor evidencia la traducción de direcciones intermedia.

8. Pruebas y resultados

La Tabla 6 resume las pruebas realizadas sobre el sistema desplegado, con el resultado observado en cada caso.

Tabla 6. Pruebas ejecutadas y resultados observados.

N.º	Prueba	Resultado observado
1	Resolución DNS	El nombre público resolvió a la dirección de la instancia
2	Conectividad SSH	Conexión establecida en el puerto 22; sesión iniciada
3	Instalación de Docker	Servicio activo, motor 25.0.16
4	Construcción de la imagen	Finalizada; imagen smart-telemetry:v1 disponible
5	Contenedor y panel local	Contenedor en ejecución; respuesta HTTP 200
6	TCP externo 5001	Conexión establecida desde un equipo externo
7	TCP externo 8080	Panel abierto correctamente en el navegador
8	Cinco nodos simultáneos	NODE01 a NODE05 reportando; cinco nodos activos
9	Consulta de nodos activos	`OK|NODES|5 con los cinco identificadores`

N.º	Prueba	Resultado observado
10	Consulta de nodo específico	`OK|NODE|NODE03 con sus cuatro variables y ACTIVE|1`
11	Última medición	`OK|LATEST con la lectura más reciente`
12	Alertas y estado general	`OK|ALERTS|100 y OK|SYSTEM|NODES|5|ALERTS|100`
13	Dos operadores	Ambos equipos obtuvieron respuesta usando el mismo nombre DNS
14	Alerta controlada	Se generó DEMO_ALERT,TEMP_HIGH,45.00
15	Tráfico UDP	100 transmitidos, 100 recibidos, 0 pérdidas estimadas
16	Recuperación	De 5 nodos a 0 al detenerlos y de nuevo a 5 al reiniciarlos
17	Captura con Wireshark	567 paquetes con los tres tipos de tráfico (sección 7)

8.1. Prueba de mensajes transmitidos, recibidos y perdidos

Se ejecutó una prueba que transmite un número conocido de datagramas y compara ese total con el contador interno del servidor, obtenido mediante el comando GET_STATS. El resultado fue de 100 mensajes transmitidos, 100 recibidos y ninguna pérdida estimada.

Este resultado corresponde únicamente a esa ejecución y no implica que UDP garantice la entrega. La ausencia de pérdidas se explica por el bajo volumen, el intervalo entre envíos y unas condiciones de red favorables en ese momento; bajo congestión o con tasas superiores el resultado podría diferir. La prueba debe ejecutarse además sin otros nodos activos para aislar correctamente el conteo.

8.2. Generación de alertas

Las alertas se disparan cuando la temperatura supera 40, la humedad 90, el consumo energético 800 o la vibración 8. Dado que los nodos generan valores aleatorios dentro de rangos que atraviesan esos umbrales, el sistema produce alertas de forma continua durante su operación normal: en las pruebas, el búfer de cien posiciones se llenaba por completo en menos de dos minutos con cinco nodos activos.

Para demostrar la generación controlada de una alerta concreta se envió además un datagrama con una temperatura de 45 grados desde un identificador reservado, y la consulta posterior mostró explícitamente la alerta correspondiente. Esa comprobación acredita la recepción y el procesamiento de esa entrega concreta, no una garantía general de entrega de UDP.

8.3. Recuperación ante desconexión

Se comprobó el comportamiento del sistema ante la interrupción del servicio. Al detener los cinco nodos, las consultas pasaron a reportar cero nodos activos sin que el servidor se viera afectado; al reanudarlos, los cinco volvieron a registrarse automáticamente. En una prueba complementaria realizada en entorno local se detuvo el servidor mientras los nodos seguían ejecutándose: estos continuaron emitiendo datagramas sin reportar error alguno, comportamiento coherente con la naturaleza no confirmada de UDP, y al reiniciar el servidor el sistema se reconstruyó sin intervención manual, aunque con los contadores y el historial de alertas reiniciados por residir el estado en memoria.

9. Problemas encontrados y soluciones implementadas

9.1. Cambio de la dirección pública de la instancia

Durante el desarrollo, la dirección pública de la instancia cambió de 44.192.90.37 a 3.236.83.176 tras detenerla e iniciarla, y con ella el nombre DNS asociado. El problema obligó a repetir pruebas que habían quedado documentadas contra la dirección anterior. La solución adoptada fue consultar el nombre vigente en la consola de EC2 antes de cada sesión de trabajo y pasarlo como argumento a los clientes, lo que resulta posible porque el código no contiene direcciones fijas. Una solución definitiva requeriría asociar una dirección elástica a la instancia.

9.2. Acceso denegado desde equipos no autorizados

El grupo de seguridad restringe el acceso a las direcciones públicas de los integrantes. Al incorporarse un integrante cuya dirección no figuraba en las reglas, las conexiones fallaban sin mensaje claro. La solución consistió en consultar la dirección pública propia y solicitar su inclusión en el grupo de seguridad para los tres puertos del sistema. Conviene advertir que las direcciones residenciales son dinámicas y pueden cambiar, por lo que la comprobación debe repetirse ante cualquier fallo de conectividad inesperado.

9.3. Captura de tráfico sobre la interfaz equivocada

El equipo empleado para el análisis de red dispone de varias interfaces, entre ellas las virtuales creadas por el subsistema Linux y por Docker. Ejecutar los clientes en el subsistema Linux y capturar en su interfaz virtual habría mostrado direcciones internas en lugar del tráfico real hacia la nube. La solución fue ejecutar los clientes con el intérprete nativo del sistema anfitrión y capturar sobre el adaptador físico, verificando previamente con una consulta de enrutamiento cuál era la interfaz por la que salía el tráfico hacia el servidor.

9.4. Identificadores incompletos en las consultas

Al consultar un nodo específico introduciendo únicamente el número, el servidor respondía con el error de nodo no encontrado. No se trata de un defecto sino del comportamiento correcto del protocolo, que exige el identificador completo. Se documentó explícitamente en las instrucciones de uso para evitar la confusión, y el caso quedó registrado como ejemplo del manejo de errores del protocolo.

9.5. Terminadores de línea del script de arranque

El script de punto de entrada del contenedor, editado en Windows, contenía terminadores de línea de ese sistema que impedían su ejecución en Linux. Se corrigió convirtiendo los terminadores antes de construir la imagen. Es un problema habitual al trabajar con repositorios compartidos entre sistemas operativos distintos.

9.6. Salida del servidor no visible en los registros del contenedor

Los mensajes que el servidor imprime por consola no aparecían al consultar los registros del contenedor. La causa es que la biblioteca estándar de C utiliza almacenamiento intermedio por bloques cuando la

salida no se dirige a un terminal, de modo que los mensajes quedan retenidos hasta llenar el búfer. La solución consiste en configurar la salida en modo de línea al iniciar el programa. El problema no afecta al funcionamiento del servicio, pero dificulta el diagnóstico en el entorno desplegado.

9.7. Binarios compilados en el control de versiones

El repositorio incluía un ejecutable compilado, añadido inadvertidamente en una etapa temprana. Un binario no aporta información al historial, depende de la plataforma y se regenera con la compilación. Se añadieron las reglas correspondientes al archivo de exclusiones y se retiró el archivo del control de versiones conservándolo en disco.

10. Conclusiones

El proyecto integra en un sistema funcional los conceptos de la arquitectura TCP/IP abordados en el curso, y esa integración resultó verificable sobre tráfico real y no únicamente sobre el comportamiento observable de las aplicaciones.

La elección diferenciada de protocolos de transporte quedó justificada con datos propios. El análisis de la captura mostró que el 82 % de los paquetes TCP intercambiados no transporta datos de aplicación, sino control de conexión, mientras que la totalidad de los datagramas UDP transporta una medición. Esa medición convierte en cuantitativa una decisión que de otro modo habría sido meramente declarativa: la fiabilidad tiene un costo medible, y sólo conviene pagarlo donde la aplicación la necesita.

El diseño de un protocolo propio de capa de aplicación en texto plano resultó acertado por una razón que no se había anticipado: hizo que el contenido de la comunicación fuese directamente legible en el analizador de red, lo que permitió verificar la correspondencia entre la especificación y el tráfico efectivo. El hecho de que el analizador clasifique esos paquetes como datos genéricos, frente al reconocimiento explícito que hace de HTTP, evidencia que se trata de un protocolo ajeno a los estándares existentes.

El despliegue en la nube dentro de un contenedor demostró ser reproducible, y el uso de nombres DNS en lugar de direcciones fijas permitió que el sistema siguiera funcionando pese a que la dirección pública de la instancia cambió durante el desarrollo. Ese incidente, inicialmente un contratiempo, acabó confirmando el valor de la indirección que proporciona la resolución de nombres.

Entre las limitaciones identificadas destacan la residencia del estado en memoria, que provoca su pérdida ante cualquier reinicio, y los límites fijos del registro de nodos y del historial de alertas. Ninguna afecta a los requisitos del proyecto, pero todas serían determinantes en un despliegue real, donde la persistencia y la escalabilidad del almacenamiento constituyen requisitos de primer orden.

11. Evidencias individuales

Las capturas de pantalla que siguen fueron tomadas en los equipos de los integrantes y permiten verificar la participación de cada uno. En todas ellas se muestran, en la misma terminal, los comandos de identificación seguidos de la ejecución correspondiente.

11.1. Juan José Díaz

Aporte: captura y análisis de tráfico de red con Wireshark, documentado en la sección 7. Equipo JUANJOSEDIAZ, usuario juanjosediaz\jjdia.

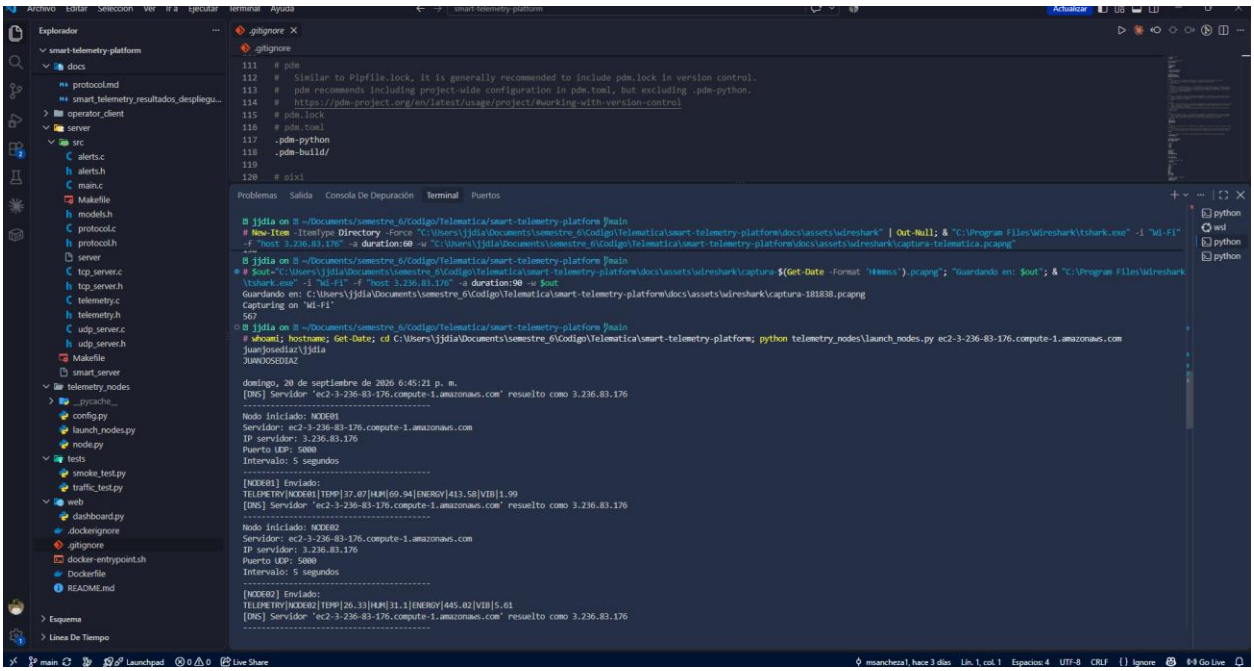


Figura 10. Ejecución de los cinco nodos de telemetría contra el servidor desplegado. Se identifican el usuario, el hostname, la fecha, el entorno de desarrollo y la resolución DNS del nombre de la instancia.

La ejecución produce la línea de resolución del nombre público de la instancia, que documenta la localización del servidor mediante DNS y no mediante una dirección fija, seguida de los mensajes de telemetría efectivamente transmitidos.

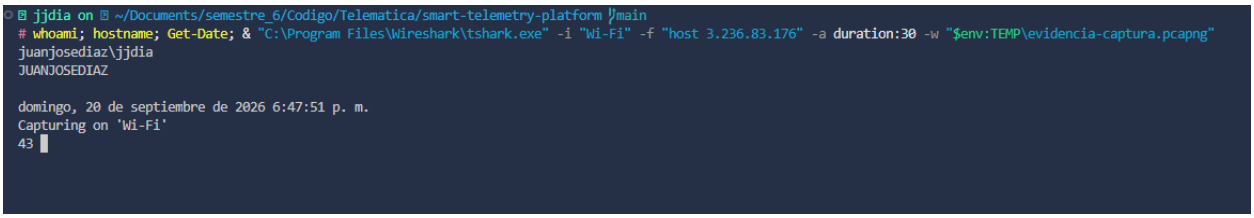


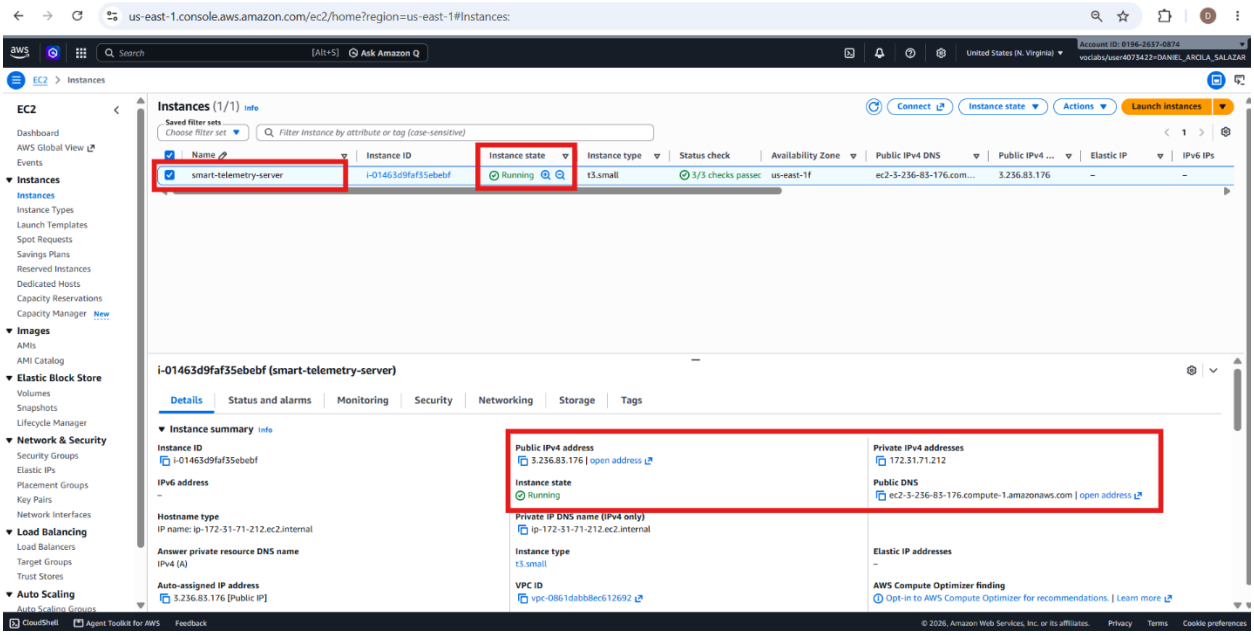
Figura 11. Captura de tráfico en vivo sobre el adaptador físico, filtrada a la dirección del servidor desplegado, con el contador de paquetes en ejecución.

El filtro de captura restringe el registro al tráfico intercambiado con la instancia, y la interfaz seleccionada es el adaptador físico, lo que confirma que se trata de tráfico real a través de Internet y no de tráfico local.

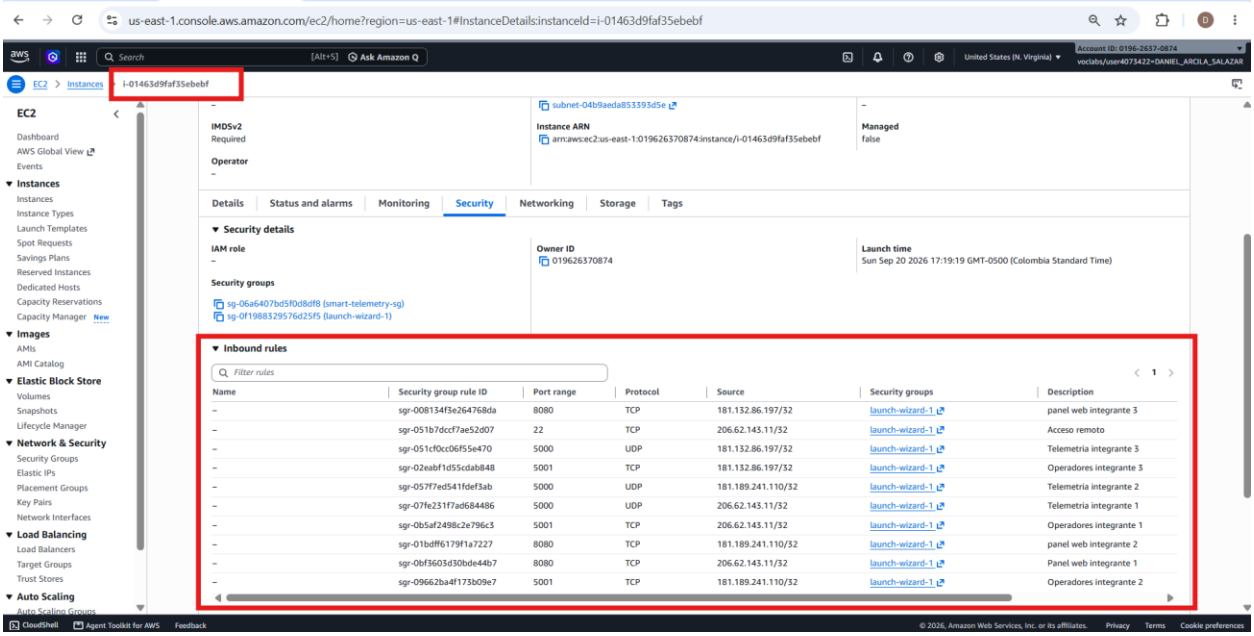
11.2. Daniel Arcila

Aporte: despliegue en AWS, configuración de Docker y pruebas funcionales sobre la instancia, documentado en la sección 6.

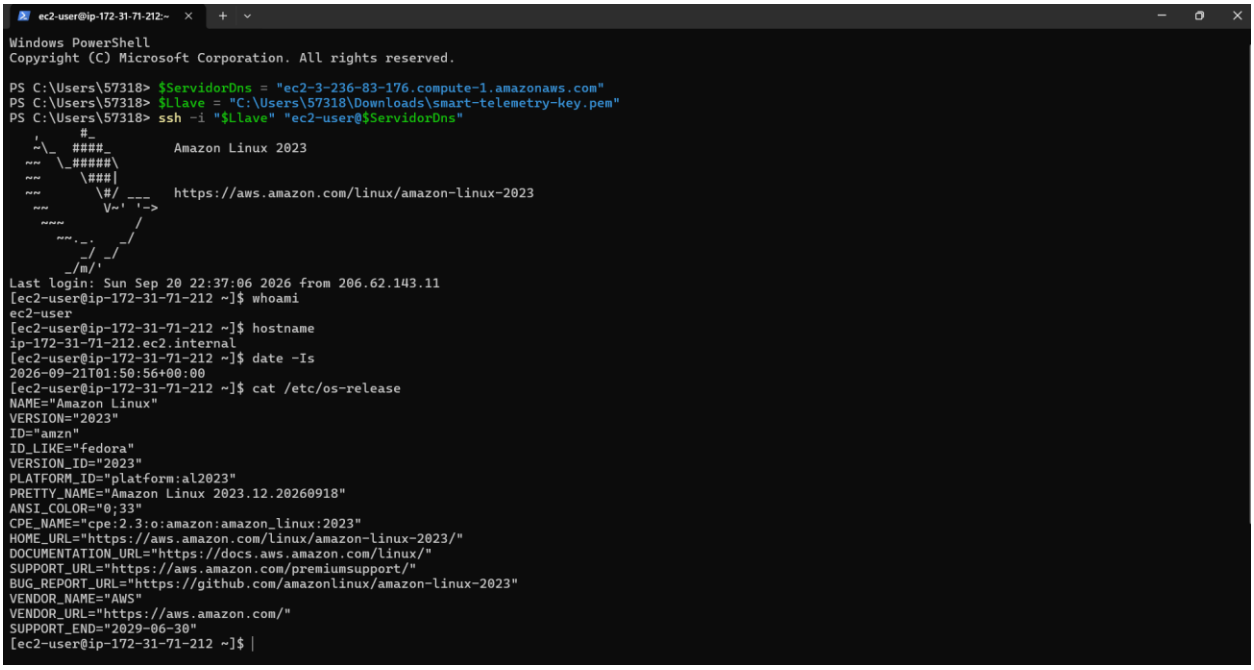
Se verifica que la instancia smart-telemetry-server se encuentra en estado Running, con las comprobaciones de estado correctas y una dirección IP pública asignada.



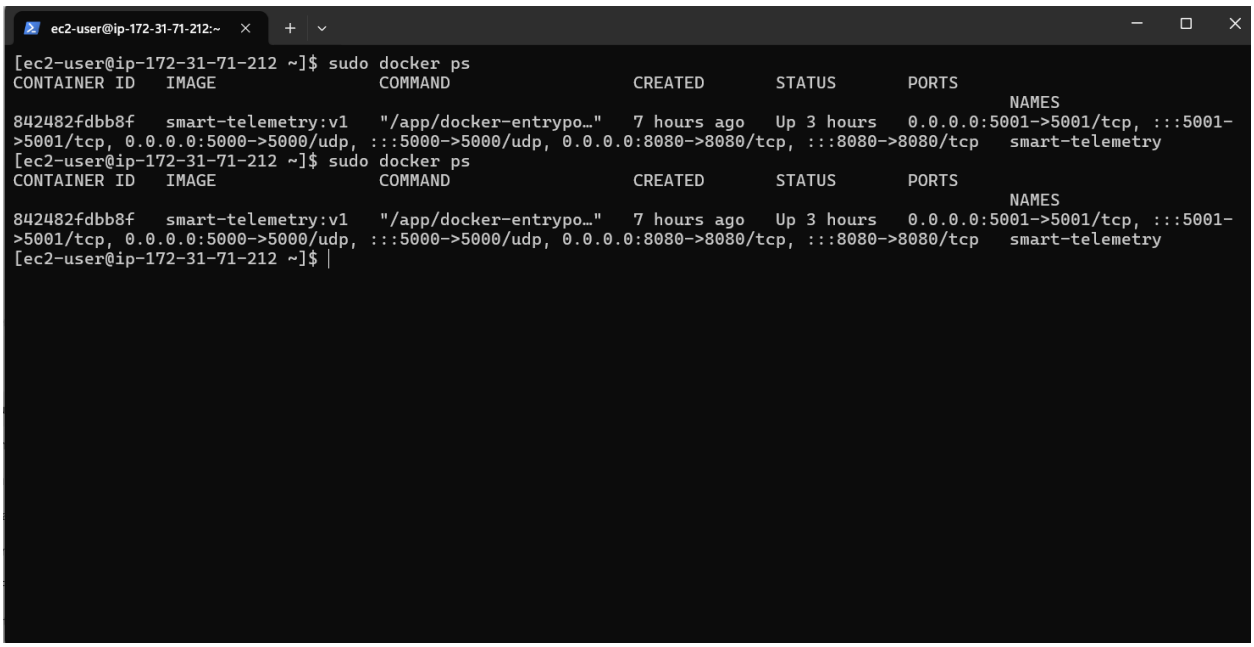
Se evidencian las reglas de entrada que permiten el acceso SSH (22/TCP), la recepción de telemetría (5000/UDP), las consultas de operadores (5001/TCP) y el acceso al panel web (8080/TCP).



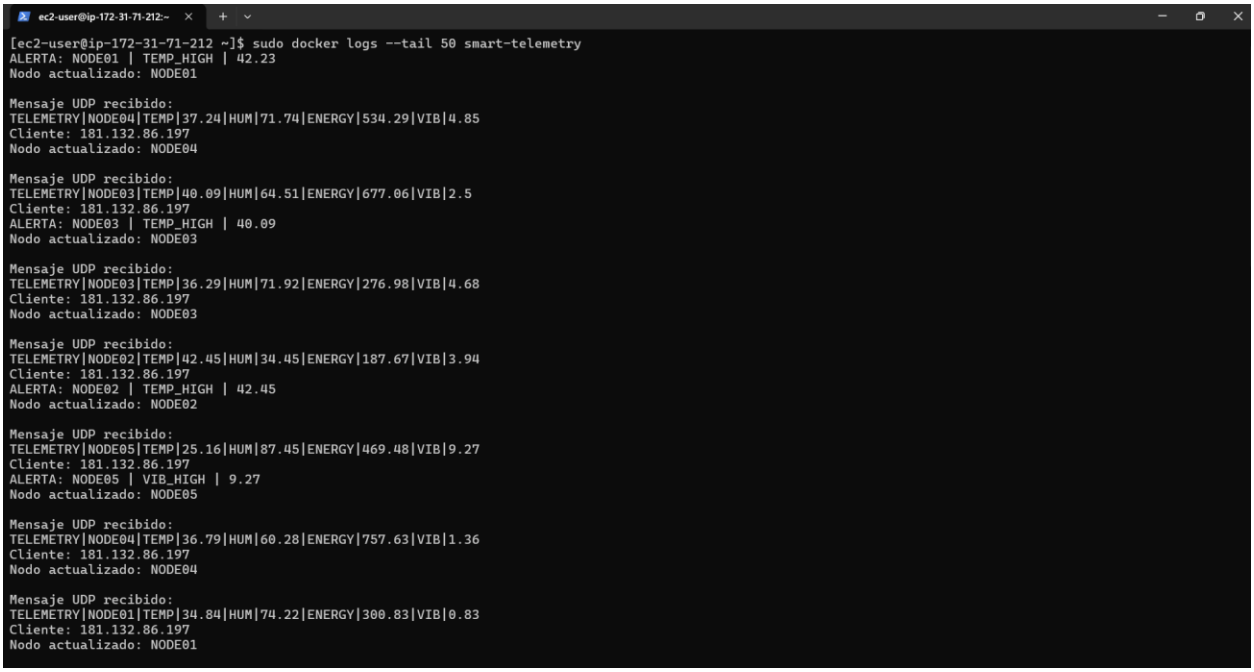
Se demuestra el acceso remoto exitoso a la instancia EC2 mediante SSH, verificando el usuario, el nombre del servidor, la fecha y el sistema operativo Amazon Linux 2023.



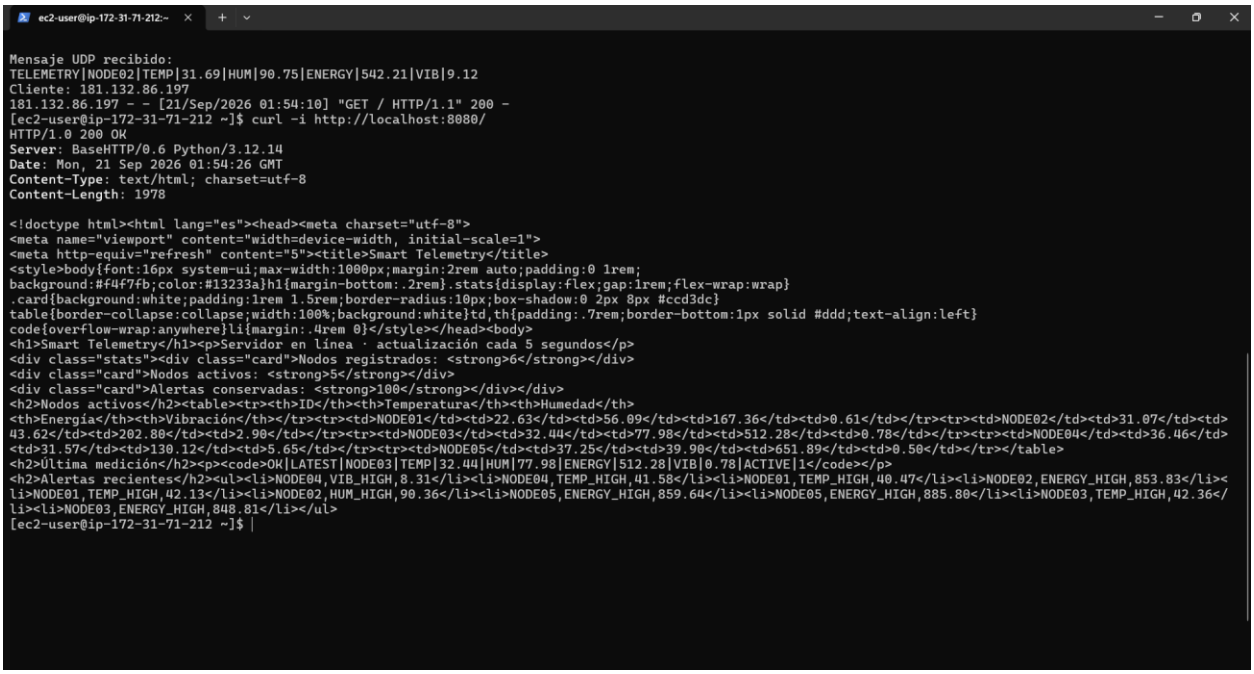
El comando docker ps confirma que el contenedor smart-telemetry se encuentra activo y tiene publicados los puertos 5000/UDP, 5001/TCP y 8080/TCP.



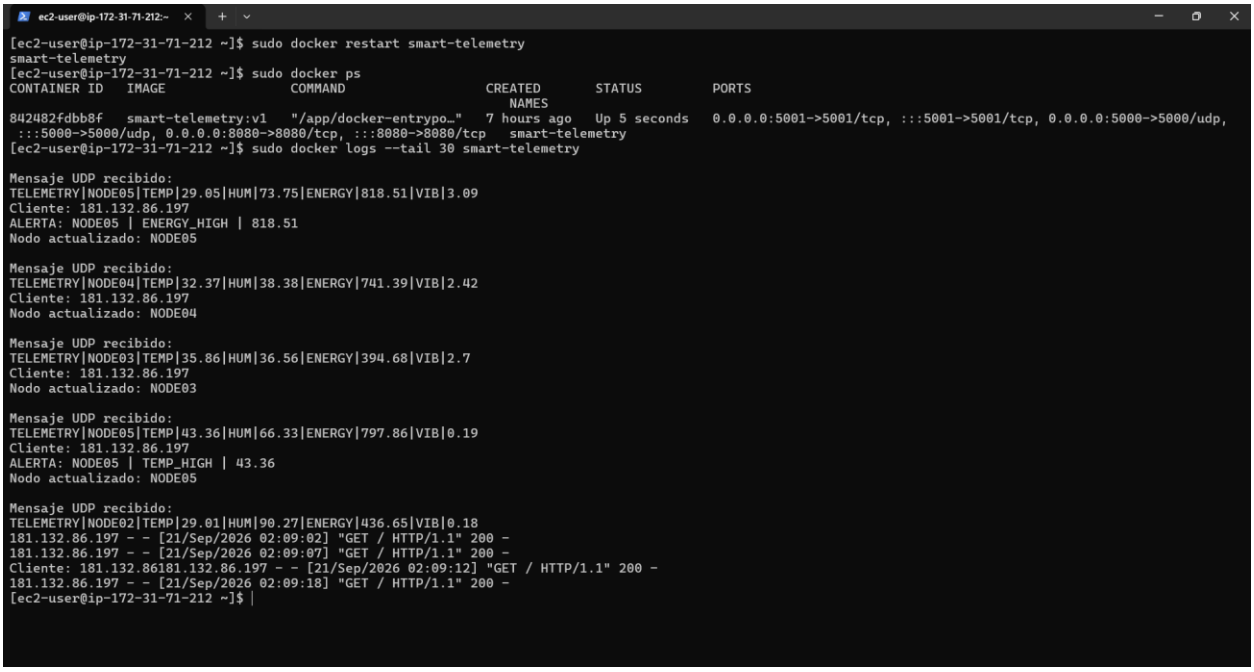
Se muestran los registros de Docker, evidenciando la recepción de mensajes UDP, la actualización de los nodos y la generación de alertas.



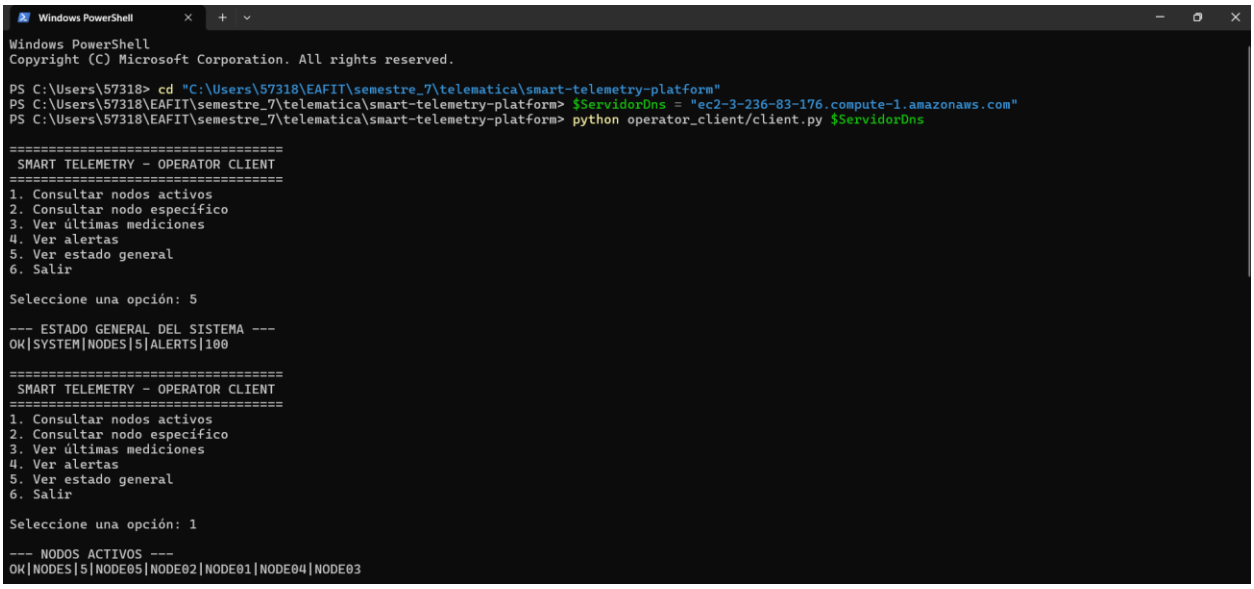
La solicitud HTTP realizada desde EC2 devuelve el código 200 OK, confirmando que el panel web responde correctamente en el puerto 8080.



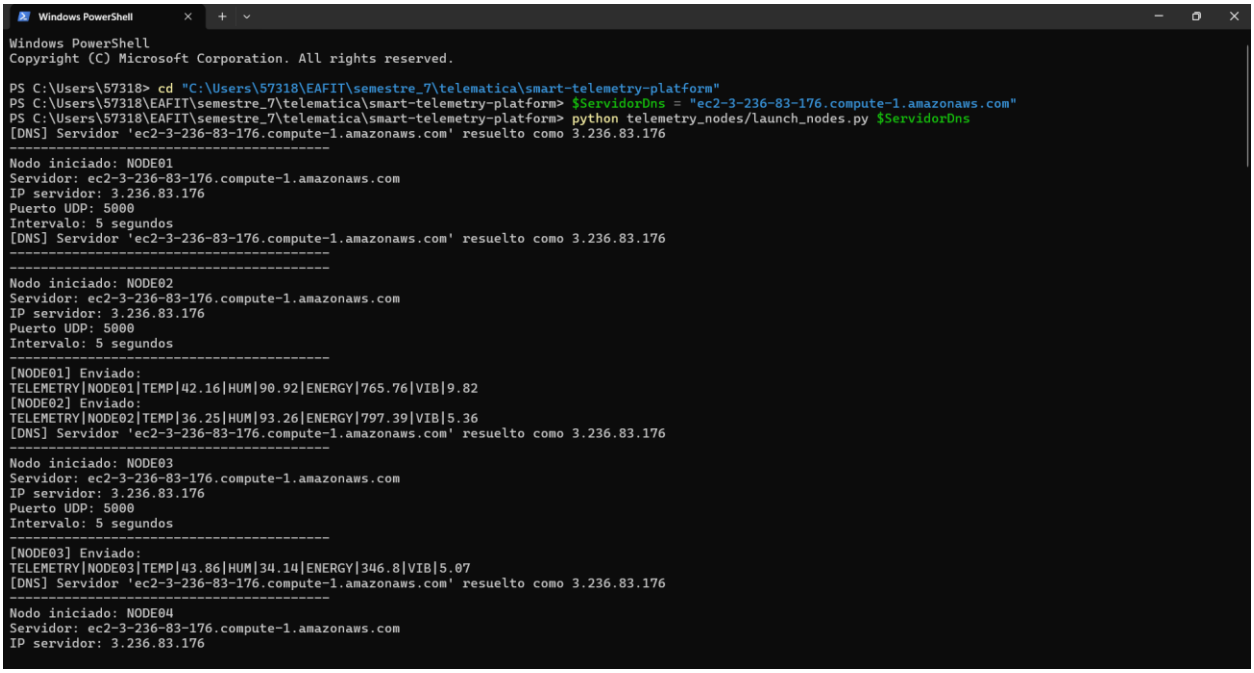
Se ejecuta docker restart y se comprueba que el contenedor vuelve al estado Up, conserva los tres puertos publicados y continúa procesando mensajes.



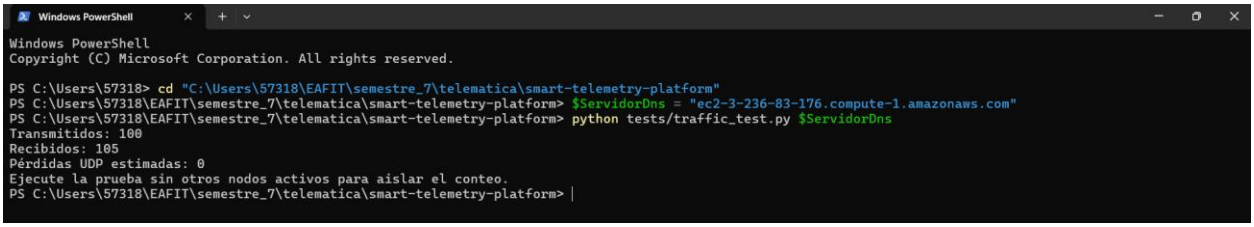
Se verifica la comunicación TCP con el servidor mediante consultas del estado general y de los nodos activos, obteniendo respuestas satisfactorias con cinco nodos registrados como activos.



Se evidencia la ejecución de los nodos Python, que transmiten periódicamente mediciones al servidor EC2 mediante UDP por el puerto 5000.



Se transmitieron 100 mensajes y se contabilizaron 105 recepciones, con cero pérdidas estimadas reportadas.



11.3. Mariana Sánchez

Aporte: desarrollo del servidor central en C, los nodos de telemetría y el cliente operador, documentado en las secciones 3, 4 y 5.

1. Identificación del equipo

Esta evidencia permite identificar el computador desde el cual se realizó la prueba y relacionarlo con la participación de la estudiante.

Comandos utilizados

whoami

hostname

date

Evidencia



Figura 1. Identificación del equipo y fecha de la prueba

2. Ejecución del nodo de telemetría

Se ejecutó el nodo de telemetría desarrollado en Python. El nodo genera periódicamente mediciones de temperatura, humedad, energía y vibración y las envía al servidor mediante UDP.

Comando utilizado

python telemetry_nodes/node.py NODE01

Evidencia

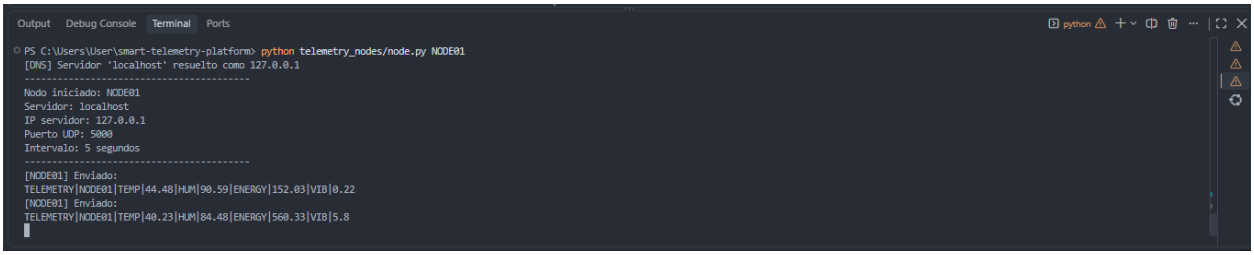


Figura 2. Ejecución del nodo NODE01 y envío de telemetría

3. Recepción y procesamiento de telemetría en el servidor

El servidor central recibe los mensajes enviados por los nodos mediante UDP. Posteriormente, el protocolo de aplicación es interpretado y los datos son convertidos a la estructura interna NodeTelemetry.

Evidencia

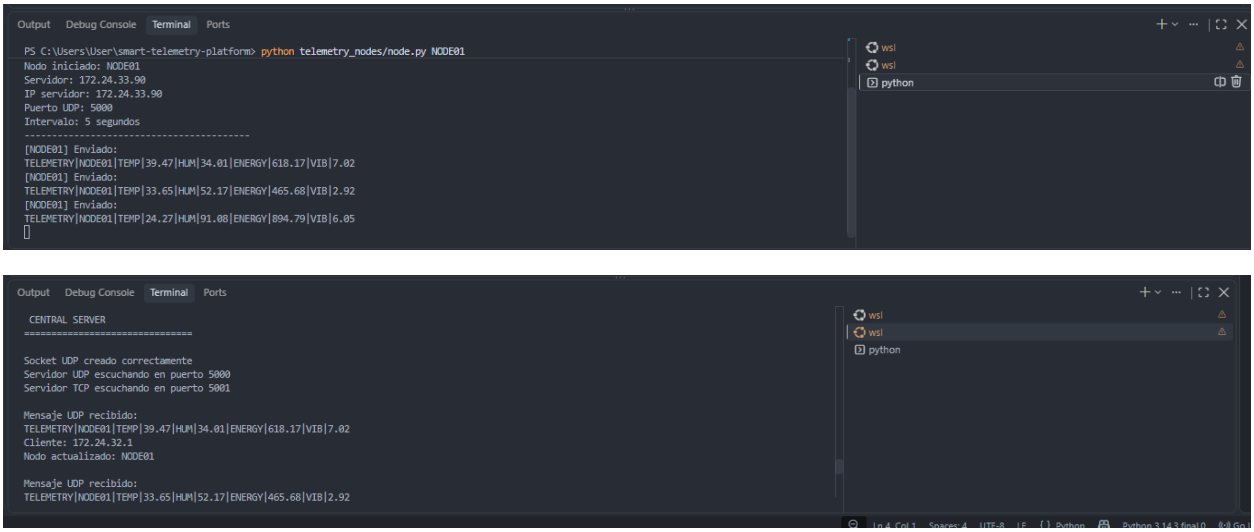


Figura 3. Recepción y procesamiento de un mensaje UDP

4. Actualización de información del nodo

Después de interpretar el mensaje, el servidor actualiza la información correspondiente al nodo.

Evidencia

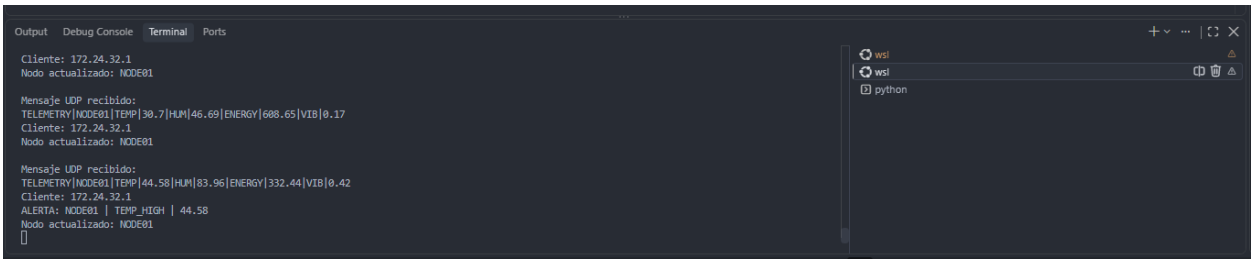


Figura 4. Nodo actualizado correctamente

5. Generación de alertas

El servidor analiza las mediciones recibidas y genera una alerta cuando una variable supera los límites definidos.

Las condiciones implementadas son:

Variable Condición Tipo de alerta

Temperatura TEMP > 40 TEMP_HIGH
Humedad HUM > 90 HUM_HIGH
Energía ENERGY > 800 ENERGY_HIGH
Vibración VIB > 8 VIB_HIGH

Evidencia



Figura 5. Generación de una alerta

6. Comunicación con el cliente operador

El cliente operador se desarrolló en Python y utiliza TCP para realizar consultas al servidor.

Comando utilizado

python operator_client/client.py

Evidencia

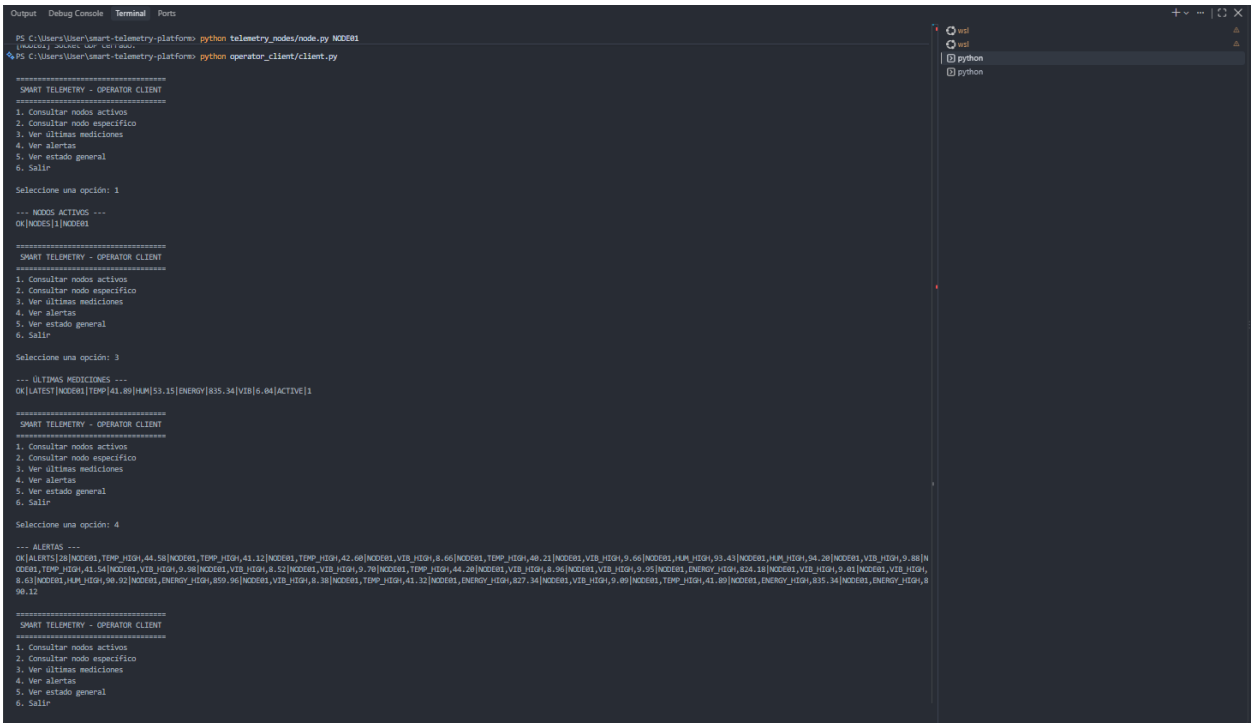




Figura 6. Cliente operador conectado al servidor

7. Operación simultánea de nodos

La plataforma debe permitir la ejecución simultánea de cinco o más nodos de telemetría.

Evidencia

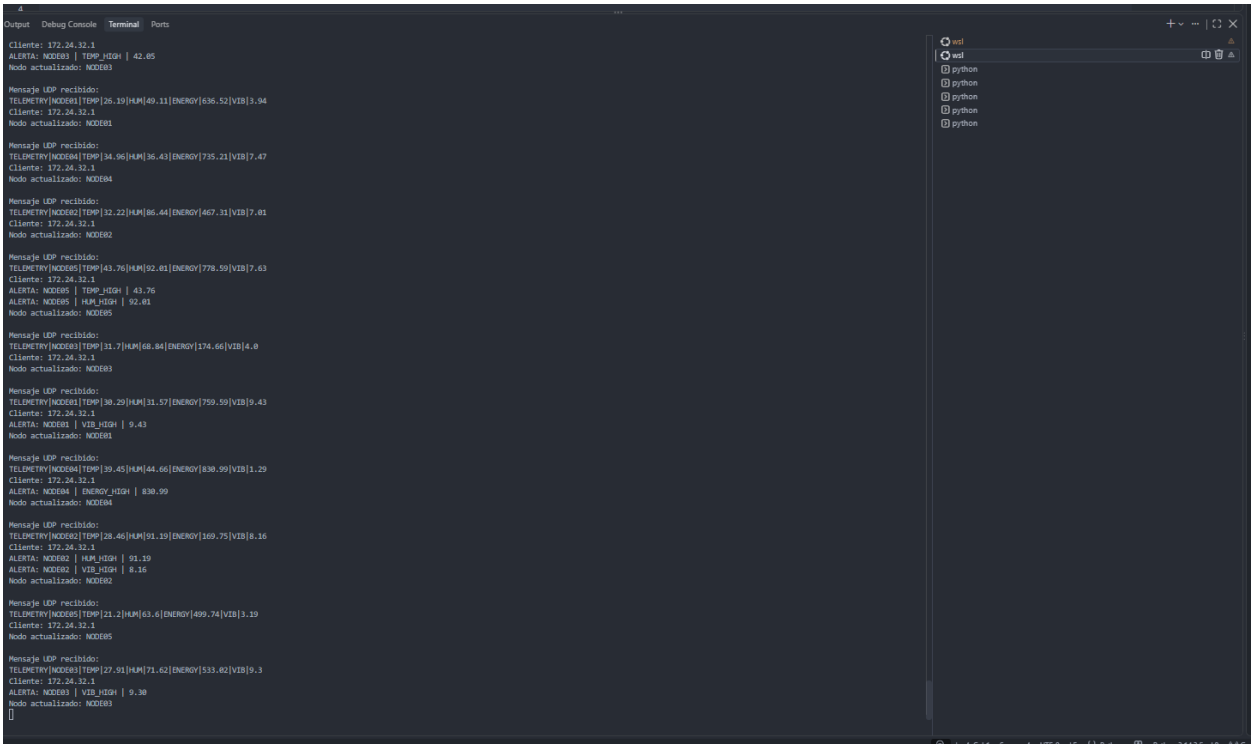




Figura 7. Ejecución simultánea de los nodos

8. Evidencia de Docker

El servidor debe ejecutarse dentro de un contenedor Docker.

Comandos utilizados

docker images

docker ps

Evidencia



Figura 8. Imagen y contenedor Docker